SafeLane-RL: Teaching a Car to Drive and Merge Safely with Reinforcement Learning

Gokul Bansal (B24CM1025) · Likhita (B24CM1043)
AI Course, Assignment 2 (Real-Life Use Cases of RL), IIT Jodhpur
GitHub: `https://github.com/Osptium-Invincible/safelane-rl`

0%	**97–99%**	**94%**	**10.5 h**
crashes on the official highway test (150 episodes)	successful merges from the on-ramp (plain PPO: 51%)	merges in **2**× the traffic it trained on (rule-based driver: 72%)	CPU-hours of training for all 61 runs, on one laptop (no GPU)

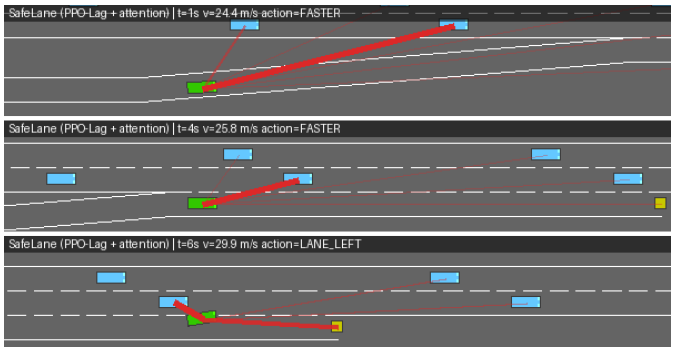


Figure 1: SafeLane merging (frames from our GIF). Green = our car, blue = traffic, yellow = end-of-ramp barrier; red lines = where the attention network is "looking". Top: on the ramp, speeding up. Middle: barrier ahead, choosing a gap. Bottom: moving left into the gap.

1. The problem

Changing lanes and merging from an on-ramp are among the riskiest highway manoeuvres: the driver must judge gaps in moving traffic, adjust speed, and commit before the ramp ends. Hand-written rule-based drivers are predictable but rigid, and in heavy traffic they either refuse to merge or merge unsafely. We train a car to make these **tactical decisions** (which lane, faster or slower) *purely by trial and error* with reinforcement learning (RL); simple controllers do the steering and throttle, as in real self-driving cars. Our main idea: treat safety as a **budget** the agent must respect, not as one more term in the reward. We call the agent **SafeLane**.

2. Simulator and scenarios

We use **highway-env** 1.12.1, the standard RL simulator for highway driving. The other cars react to us: **IDM** sets their speed (keep a safe gap to the car ahead) and **MOBIL** decides their lane changes. We made traffic more varied: each car has its own reaction gap (0.5–2.5 s) and politeness, and drivers can be normal, aggressive or cautious.

- **Highway:** train on 3 lanes with 20 cars; test on the official `highway-v0` (4 lanes, 50 cars, 40 s).
- **RampMerge (built by us):** our car starts on the on-ramp and must merge within 80 m before the ramp ends at a barrier (Fig. 1). *Success* = merged and passed the merge zone without crashing. (highway-env's own merge scenario puts our car on the main road, so it cannot measure this.) Along the way we found and fixed a highway-env bug: a car on the ramp "saw" main-road cars as if they were in front of it.

3. RL formulation

One **step** = one driving decision, made once per second. One **episode** = one drive; it ends early on a crash.

State S	A table of nearby cars: one row per car (ours first, then the nearest), with position and speed *relative to us*, scaled to $[-1, 1]$. Plain networks: 5 cars $\times$ 5 numbers $= 25$. Attention network: 15 cars $\times$ 7 numbers $= 105$. Only cars within 200 m are visible.
Action A	5 discrete choices: LANE-LEFT, IDLE, LANE-RIGHT, FASTER, SLOWER (target speeds 10–30 m/s).
Reward R	*Highway:* $r = \text{scale}_{[0,1]}(-1{\cdot}\text{crash} + 0.1{\cdot}\text{lane} + 0.4{\cdot}\text{speed})$; "speed" maps 20–30 m/s to 0–1, "lane" rewards keeping right. *Ramp:* $r = -1 \cdot \text{crash} + 0.2 \cdot \text{speed} + 1 \cdot \text{merged}$. A crash also ends the episode, losing all future reward.
Cost c	$c = [\text{TTC} < 2\,\text{s}] + [\text{headway} < 1\,\text{s}] + 10 \cdot [\text{crash}]$, where TTC (time-to-collision) = gap ÷ closing speed and headway = gap ÷ our speed. **Budget:** average episode cost $\leq d = 2$.
Discount	$\gamma = 0.9$: the agent plans about 10 s ahead.

4. Our agent: SafeLane

SafeLane = three ingredients (Fig. 3):

1. **PPO**: a policy-gradient algorithm that directly learns how likely each action should be, improving it in small, safe steps (8 simulators in parallel).
2. **Attention network** (written by us): every car is encoded by the same small network, and our car "asks" which cars matter right now. It handles any number of cars and its weights can be drawn (red lines in Fig. 1). 46.7k parameters vs 146k for a plain network.
3. **Safety budget** (PPO-Lagrangian, a *constrained MDP*): maximise reward *such that* average episode cost $\leq d$. The agent maximises $r - \lambda c$, and the safety weight λ is *learned*: over budget $\Rightarrow \lambda$ goes up (safety matters more); under budget $\Rightarrow \lambda$ goes down (speed matters again). A PID controller updates λ every 1,024 steps. In training, the episode cost started at 10–13 and was pushed down to the budget of 2; λ rose while the agent was unsafe, then fell.

Why not just a big crash penalty? Its right size is unknown (too small: crashes; too large: the car crawls) and must be re-tuned per task. A budget is set once, in understandable units.

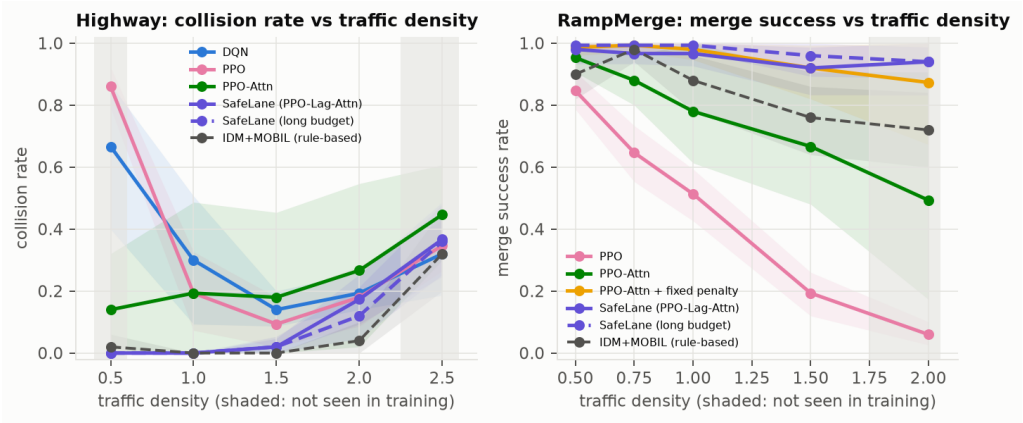


Figure 2: Lighter and heavier traffic than in training (grey shading = never seen in training). Left: highway crash rate (lower is better). Right: merge success (higher is better). SafeLane is purple.

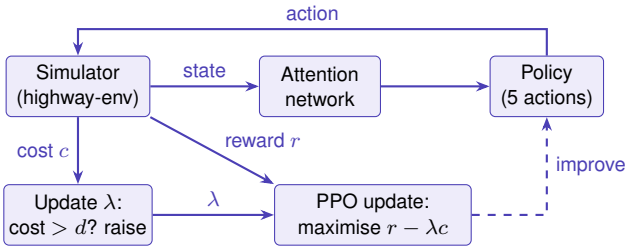


Figure 3: How SafeLane learns.

5. What we compared, and how

- **Baselines:** random; constant speed; a **rule-based driver** (IDM+MOBIL; on the ramp it matches speed and merges into gaps ≥ 0.8 s).
- **Value-based RL:** DQN; our D3QN (Double + Dueling + 3-step DQN); QR-DQN.
- **PPO family:** PPO; PPO + attention; the same with a *fixed* penalty ($\lambda = 0.1$); MaskablePPO, where a hand-written rule blocks risky actions (a "shield").
- **Ours:** SafeLane, and SafeLane-Mixed (one agent for both tasks).

Training: traffic density and driver types are randomised every episode, so the agent cannot memorise one situation. 50k steps for DQN-type agents, 120k for PPO-type; a "long" round gives SafeLane 400k. **3 random seeds** per method. **Testing:** 50 episodes per agent per scenario on unseen seeds, across **26 scenarios** (lighter/heavier traffic incl. levels never seen in training, aggressive/cautious drivers, sensor noise, hidden cars, delayed sensors): 49,250 test episodes.

6. Results

Highway. SafeLane never crashed in 150 episodes on the official benchmark and had the highest score (Table 1). The rule-based driver was also crash-free but scored less (30.1 vs 33.0): on the highway, RL's gain is *efficiency*, not safety. Attention alone is not enough: PPO + attention without the budget crashed 18% of the time.

Merging. Without a budget, PPO agents drive at about 28 m/s and crash in 22–49% of merges. SafeLane succeeds 96.7% of the time, and 99.3% after longer training.

Budget vs fixed penalty. A well-chosen fixed penalty ties at normal traffic (98.0 vs 96.7%), but drives more riskily (safety cost 3.2 vs 1.2; near-misses 18.7 vs 6.7%) and drops more in heavy traffic (87 vs 94%). SafeLane found its own λ (0.12–0.47 per task and seed); nothing was hand-tuned.

Shield. The shield backfires in heavy traffic: 43 of MaskablePPO's 45 crashes at 2× traffic hit the end-of-ramp barrier. The rule blocks risky merges, but the agent never learns to slow down and *wait* for a gap.

Table 1: Main results: average of 3 seeds × 50 episodes. Crash and success in %; score = average total reward (higher is better). "–" = not trained for that task.

	Highway		Merge success	
Agent	crash	score	normal	2× traffic
Random	72.0	12.9	30.0	20.0
Constant speed	96.0	14.2	0.0	0.0
Rule-based (IDM+MOBIL)	0.0	30.1	88.0	72.0
DQN	8.7	30.1	–	–
D3QN (ours)	36.0	25.0	87.3	60.7
QR-DQN	14.7	29.2	–	–
PPO	33.3	28.0	51.3	6.0
PPO + attention	18.0	29.7	78.0	49.3
+ fixed penalty	–	–	98.0	87.3
MaskablePPO (shield)	–	–	88.7	70.0
SafeLane	0.0	33.0	96.7	94.0
SafeLane (long)	0.7	32.8	99.3	94.0
SafeLane-Mixed (long)	3.3	31.0	98.7	82.0

Unseen traffic (Fig. 2). At 2× ramp traffic SafeLane still merges 94% of the time (PPO 6%, rule-based 72%). On a highway with half the usual traffic, plain-network agents crash 67–86% of the time by speeding into a lone slower car; SafeLane crashes 0%.

What it looks at. About 75% of SafeLane's attention goes to the 3 nearest cars, the ones around the gap it is merging into.

7. Limitations and next steps

- **Noisy sensors** are every learned agent's weak point: with position noise $\sigma = 0.1$ SafeLane crashes in 29% of highway episodes. All agents trained on perfect inputs; the rule-based driver reads the true state, so noise cannot affect it.
- In **very heavy highway traffic** (2×) the rule-based driver crashes less (4 vs 17%).
- The budget holds *on average*, not in every episode; the simulator is 2-D; 3 seeds per method.

Next: train with sensor noise, add a safety filter, use real traffic data. **Run it:** install steps are in the README; trained models are in `runs/`; `make check` re-verifies every number in this report.